

PRACTICAL 9 - SESSION 9

Task 1 :- Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
GNU nano 7.2 task1_file_copy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *source, const char *destination) {
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    src = open(source, O_RDONLY);

    if (src == -1) {
        perror("open source");
        exit(1);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1) {
        perror("open destination");
        close(src);
        exit(1);
    }

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read) {
            perror("write");
            close(src);
            close(dest);
            exit(1);
        }
    }
}
```

```
    }

    close(src);
    close(dest);
}

void library_copy(const char *source, const char *destination) {
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes;

    src = fopen(source, "r");

    if (src == NULL) {
        perror("fopen source");
        exit(1);
    }

    dest = fopen(destination, "w");

    if (dest == NULL) {
        perror("fopen destination");
        fclose(src);
        exit(1);
    }

    while ((bytes = fread(buffer, 1, BUFFER_SIZE, src)) > 0) {
        fwrite(buffer, 1, bytes, dest);
    }

    fclose(src);
    fclose(dest);
}

int main() {
    FILE *file;
    clock_t start, end;
```

```

if (file == NULL) {
    perror("source.txt");
    return 1;
}

for (int i = 0; i < 10000; i++) {
    fprintf(file,
        "This is sample data for testing file copy performance.\n");
}

fclose(file);

printf("File Copy Utility\n");
printf("-----\n");

start = clock();
low_level_copy("source.txt", "low_level_copy.txt");
end = clock();

low_level_time =
    ((double)(end - start)) / CLOCKS_PER_SEC;

start = clock();
library_copy("source.txt", "library_copy.txt");
end = clock();

library_time =
    ((double)(end - start)) / CLOCKS_PER_SEC;

printf("Low-level copy completed successfully.\n");
printf("Standard library copy completed successfully.\n");

printf("\nPerformance Comparison:\n");
printf("Low-level I/O time      : %.6f seconds\n",
    low_level_time);
printf("Library I/O time       : %.6f seconds\n",
    library_time);

return 0;
}

```

The output shows that the file was copied successfully using both low-level and standard library I/O, followed by a comparison of their execution

```

root@HP:~/practical_session9# gcc task1_file_copy.c -o task1_file_copy
root@HP:~/practical_session9# ./task1_file_copy
File Copy Utility
-----
Low-level copy completed successfully.
Standard library copy completed successfully.

Performance Comparison:
Low-level I/O time      : 0.001336 seconds
Library I/O time       : 0.001896 seconds
root@HP:~/practical_session9# |

```

times.

Task 2 :- Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shell tools implement I/O redirection.

```
GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input_file;
    int output_file;
    int saved_stdin;
    int saved_stdout;
    char buffer[100];

    printf("I/O Redirection Demonstration\n");
    printf("-----\n");

    FILE *file = fopen("input.txt", "w");

    if (file == NULL) {
        perror("input.txt");
        return 1;
    }

    fprintf(file, "First line from redirected input.\n");
    fprintf(file, "Second line from redirected input.\n");
    fclose(file);

    input_file = open("input.txt", O_RDONLY);

    if (input_file == -1) {
        perror("input.txt");
        return 1;
    }

    output_file = open("redirected_output.txt",
```

```

output_file = open("redirected_output.txt",
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

if (output_file == -1) {
    perror("redirected_output.txt");
    close(input_file);
    return 1;
}

saved_stdin = dup(STDIN_FILENO);
saved_stdout = dup(STDOUT_FILENO);

dup2(input_file, STDIN_FILENO);
dup2(output_file, STDOUT_FILENO);

close(input_file);
close(output_file);

printf("Reading data from redirected standard input:\n");

while (fgets(buffer, sizeof(buffer), stdin) != NULL) {
    printf("%s", buffer);
}

fflush(stdout);

dup2(saved_stdin, STDIN_FILENO);
dup2(saved_stdout, STDOUT_FILENO);

close(saved_stdin);
close(saved_stdout);

printf("Standard input and output restored successfully.\n");

return 0;

```

The output demonstrates successful input and output redirection, with the redirected data stored in the output file and verified using

cat.

```

root@HP:~/practical_session9# gcc task2_io_redirection.c -o task2_io_redirection
root@HP:~/practical_session9# ./task2_io_redirection
I/O Redirection Demonstration
-----
Standard input and output restored successfully.
root@HP:~/practical_session9# cat redirected_output.txt
Reading data from redirected standard input:
First line from redirected input.
Second line from redirected input.
root@HP:~/practical_session9# |

```

Observation :- File copying and I/O redirection were successfully implemented and tested using low-level system calls and file descriptors.